

COMPICK

admin 코드 리뷰 기능 설명서

회원 · 구매 · 관리자 기능별 구현 분석

분석 대상	D:\2026WMLPWCOMPICK\admin
작성 기준	2026-08-12 소스 및 기존 테스트 결과

목차

- 0 시스템 개요
- 1 회원 기능
 - 1-1 로그인/회원가입
 - 1-2 회원정보(로그인정보, 배송지)
- 2 구매 기능
 - 2-1 부품 구매
 - 2-2 견적서 작성
 - 2-3 추천 프리셋 / AI 추천
 - 2-4 장바구니
 - 2-5 주문·결제 연결 기능
- 3 관리자 페이지
- 4 종합 코드 리뷰
- 5 API·화면·파일 맵

0. 시스템 개요

0.1 기술 구성

- Java 21, Spring Boot 3.5.16
- Spring MVC + Thymeleaf 서버 렌더링
- Spring Security 폼 로그인, Remember-Me, Google OIDC
- Spring Data JPA, Oracle 운영 DB, H2 테스트 DB
- Gemini Java SDK 기반 AI 견적
- Toss Payments REST 연동
- JavaScript **fetch** 기반 상품·견적·장바구니·주문 상호작용

0.2 애플리케이션 구조

com.boot.compick 아래를 업무 도메인별 패키지로 나눴다.

member	회원, 소셜 계정, 이메일 인증, 배송지
product	상품/카테고리 조회, 검색 조건, 사양 JSON
quote	직접 견적, 프리셋, AI 견적 저장, 호환성 판단
shopping.recommendation	Gemini 요청/응답, AI 결과 파싱
cart	개별 상품과 견적 묶음 장바구니
order	주문 스냅샷, 주문 상태, 취소/반품 요청
payment	Toss 승인·취소·부분 환불, 재고와 장바구니 후처리
admin	관리자용 통합 MVC 컨트롤러와 개인정보 마스킹

일반적인 요청 흐름은 다음과 같다.

브라우저 → Controller → Service → Repository → Entity/DB → DTO/Model → Thymeleaf 또는 JSON

0.3 인증과 권한

SecurityConfig가 접근 정책을 한 곳에서 정의한다.

- 공개: 홈, 상품, 직접 견적, 프리셋, AI 견적 페이지, 가입/로그인, 중복 확인, 이메일 인증, 정적 파일
- 로그인 필요: 장바구니, 마이페이지, 배송지, 주문, 결제와 관련 API
- 관리자 필요: /admin/, /api/admin/

- API 비인증 요청은 401, 일반 페이지 비인증 요청은 `/login`으로 이동
- 로그아웃 시 세션을 무효화하고 `JSESSIONID`, `remember-me` 쿠키 삭제
- 폼과 AJAX 요청은 Spring Security CSRF 토큰을 사용

0.4 핵심 데이터 관계

- 회원 1명은 배송지 여러 개와 소셜 계정 여러 개를 가질 수 있다.
- 회원 1명은 장바구니 1개를 사용하며, 장바구니에는 개별 상품 항목과 견적 항목이 따로 저장된다.
- 견적은 견적 항목 여러 개를 가지며 유형은 사용자 견적, 프리셋, AI 견적으로 구분된다.
- AI 추천 기록은 생성된 견적과 연결되고 요청/응답 JSON을 보관한다.
- 주문은 결제 시점의 상품명·가격·배송지 등을 스냅샷으로 보관하여 원본 상품 변경의 영향을 받지 않게 설계됐다.
- 결제는 주문과 연결되며 승인·취소 상태와 외부 결제 키를 보관한다.

0.5 설정과 실행 의존성

운영 실행에는 Oracle 연결 정보가 필요하다. 선택 기능별로 Google OAuth, Gemini, Toss Payments, SMTP 환경 변수가 필요하다. 비밀키는 환경 변수로 주입하도록 되어 있으나 DB 비밀번호에 기본값이 있어 제거가 권장된다. JPA는 `ddl-auto=validate`이므로 스키마가 SQL 문서와 맞지 않으면 시작에 실패한다.

1. 회원 기능

1-1. 로그인/회원가입

사용자 기능

- 아이디/비밀번호 로그인과 로그인 상태 유지
- Google OIDC 로그인
- 아이디·이메일 중복 확인
- 이메일 6자리 인증 후 회원가입
- 아이디 찾기와 비밀번호 재설정
- 로그아웃

일반 회원가입 처리 흐름

1. `/members/signup`에서 가입 폼을 표시한다.
2. 브라우저가 `/api/members/check-login-id`, `/api/members/check-email`로 중복 여부를 확인한다.
3. `/api/email-verifications/send`가 인증번호를 만들고 해시만 DB에 저장한 뒤 메일을 보낸다.
4. `/api/email-verifications/confirm`이 유효기간, 사용 여부, 시도 횟수와 번호를 검증한다.
5. 가입 POST에서 Bean Validation, 비밀번호 확인, 개인정보 동의를 검사한다.
6. `MemberService.join`이 인증 완료 상태를 다시 확인하고 아이디/이메일 중복을 재검사한다.
7. 비밀번호를 BCrypt로 해시하여 `Member`를 저장하고 인증 기록을 사용 처리한다.

인증번호는 5분 유효, 1분 재발송 제한, 최대 5회 확인 제한이다. 평문 인증번호는 메일 전송 순간에만 사용하고 DB에는 BCrypt 해시를 저장한다.

로그인 처리 흐름

`GET /login`은 화면만 제공하고 `POST /login`은 Spring Security 필터가 처리한다. `MemberUserDetailsService`가 활성 회원을 조회하고 역할을 권한으로 변환한다. 성공 시 홈으로, 실패 시 `/login?error`로 이동한다. Remember-Me를 선택하면 별도 쿠키가 발급된다.

Google 로그인은 `CompickOidcUserService` → `SocialAccountService` 순서로 처리한다. 기존 소셜 계정이면 연결된 회원으로 로그인하고, 같은 이메일의 일반 계정이 있으면 Google 계정을 연결한다. 신규 사용자는 임의의 내부 아이디/비밀번호로 회원을 만든 뒤 `/members/social-password`에서 로컬 자격 증명 설정을 요구한다.

계정 복구

`RecoveryController`가 아이디 찾기와 비밀번호 재설정을 단계별 페이지로 제공한다. 이메일 인증 중간 상태는 HTTP 세션에 저장한다. 비밀번호 재설정은 로그인 아이디와 이메일이 같은 활성 회원인지 확인한 뒤 새 비밀번호를 BCrypt로 교체한다.

핵심 코드

페이지와 가입 처리	member/controller/MemberController.java
중복 확인 API	member/controller/MemberApiController.java
인증 메일 API	member/controller/EmailVerificationApiController.java
계정 복구	member/controller/RecoveryController.java
회원 업무 규칙	member/service/MemberService.java
이메일 인증	member/service/EmailVerificationService.java
Google 계정 연결	member/service/SocialAccountService.java
보안 정책	config/SecurityConfig.java

리뷰 메모

- 장점: 컨트롤러 확인 외에 서비스에서도 중복/이메일 인증을 다시 확인해 우회 가입을 방지한다.
- 장점: 비밀번호와 이메일 인증번호가 모두 BCrypt 해시로 저장된다.
- 위험: 복구 절차의 상태가 세션 속성 여러 개에 의존한다. 완료·실패·세션 고정 공격을 고려해 상태 객체와 만료 시각을 명시하는 편이 안전하다.
- 위험: 이메일 발송과 DB 트랜잭션의 원자성이 없다. 메일 발송 실패/DB 롤백 시 사용자 경험과 재시도 정책을 정의해야 한다.
- 품질: **EmailVerificationService**, API 컨트롤러 일부가 한 줄 전체로 작성되어 리뷰와 변경 추적이 매우 어렵다.

1-2. 회원정보(로그인정보, 배송지)

프로필과 로그인 정보

로그인 사용자는 **/mypage**에서 요약 정보를 보고 **/mypage/profile**에서 이름, 이메일, 전화번호 등 프로필을 수정한다. **/mypage/password**는 현재 비밀번호를 재검증하고 새 비밀번호/확인값 일치를 확인한다. **/mypage/withdraw**는 비밀번호 검증 후 회원 상태를 탈퇴로 변경하고 연결된 소셜 계정을 삭제한 뒤 세션을 종료한다.

회원 상태는 활성/정지/탈퇴, 역할은 사용자/관리자로 구분된다. 로그인 조회는 활성 회원만 대상으로 하므로 정지·탈퇴 사용자의 인증을 차단한다.

배송지

배송지는 화면 방식과 JSON API 방식을 모두 제공한다.

목록	GET /mypage/addresses	GET /api/addresses
등록	GET /mypage/addresses/new, POST /mypage/addresses	POST /api/addresses
수정	GET/POST /mypage/addresses/{id}/edit 계열	PUT /api/addresses/{id}
삭제	POST /mypage/addresses/{id}/delete	DELETE /api/addresses/{id}
기본 설정	화면 저장 시 선택	PATCH /api/addresses/{id}/default

AddressService.findOwned가 현재 로그인 회원 소유의 배송지만 조회하므로 다른 회원의 ID를 넣는 수평 권한 상승을 막는다. 기본 배송지를 지정하면 같은 회원의 기존 기본값을 해제한다. 기본 배송지를 삭제하면 남은 배송지 중 하나를 기본값으로 승격하는 정책이 서비스에 구현돼 있다.

핵심 데이터

- **Member:** 로그인 아이디, 비밀번호 해시, 이름, 이메일, 전화, 역할, 상태, 가입/수정 시각
- **SocialAccount:** 제공자, 제공자 사용자 ID, 회원 연결
- **EmailVerification:** 이메일, 목적, 코드 해시, 만료/확인/사용 시각, 시도 횟수
- **Address:** 수령인, 전화번호, 우편번호, 기본/상세 주소, 배송 요청, 기본 여부

리뷰 메모

- 장점: 배송지 서비스 전 구간에서 로그인 아이디를 받아 소유권을 확인한다.
- 장점: 엔티티 직접 노출을 피하는 **AddressRequest/Response** API DTO가 있다.
- 위험: 프로필 이메일 변경 시 새 이메일 소유 확인 절차가 코드상 명확하지 않다. 이메일을 로그인 복구 수단으로 쓰므로 변경 전에 재인증하는 것이 바람직하다.
- 위험: 탈퇴 시 소셜 연결만 삭제하고 주문·배송지·개인정보 보존/파기 정책은 별도로 보이지 않는다. 개인정보 정책 및 법적 보존 기간과 연결해야 한다.

2. 구매 기능

2-1. 부품 구매

기능 설명

`/products`에서 카테고리, 키워드, 가격, 제조사, 사양 조건으로 부품을 찾고 상세 모달을 열 수 있다. 상품 목록은 `ProductApiController`가 JSON으로 제공하고 `products.js`, `product-modal.js`가 화면을 갱신한다. 로그인 사용자는 상품 상세에서 개별 부품을 장바구니에 넣을 수 있다.

`ProductService`는 `ProductSpecifications`로 동적 검색 조건을 구성하고 `ProductSpecFacetRepository`로 카테고리별 필터 후보를 만든다. `specJson`은 상품 종류마다 다른 사양을 유연하게 담지만, 유효성은 애플리케이션 코드가 책임진다.

주요 API

- GET `/api/products`: 상품 검색/페이지 목록
- GET `/api/products/{category}/facets`: 카테고리별 필터 값
- GET `/api/products/detail/{productId}`: 상품 상세
- POST `/api/cart/items`: 개별 상품 장바구니 추가

리뷰 메모

- 장점: 목록 DTO와 상세 DTO를 구분해 필요한 데이터만 전송한다.
- 장점: 카테고리별 사양 필터와 JPA Specification을 분리했다.
- 위험: `specJson` 파싱 실패를 여러 곳에서 무시하여 잘못된 데이터가 조용히 빈 사양으로 보일 수 있다. 로그와 관리자 검증이 필요하다.
- 위험: 판매 상태·재고 검증이 조회, 장바구니, 주문, 결제 각 단계에서 일관되게 적용되는지 회귀 테스트가 부족하다.

2-2. 견적서 작성

기능 설명

`/quotes/new`에서 CPU, 메인보드, 메모리, GPU, 저장장치, 파워, 케이스 등 카테고리별 부품과 수량을 선택한다. `quote-builder.js`가 상품 조회, 선택 상태, 가격 합계와 화면 경고를 관리한다. 저장 요청은 `/api/cart/quotes`로 전송된다.

`QuoteService.buildAndAddToCart`는 다음을 한 트랜잭션에서 수행한다.

1. 로그인 회원과 요청 상품을 조회한다.

2. 중복/누락/수량을 검증한다.
3. **QuoteSelectionValidator**로 필수 구성과 카테고리 규칙을 검증한다.
4. 사용자 견적과 견적 항목을 저장한다.
5. 회원 장바구니에 견적 묶음을 추가한다.

호환성 관련 계산은 CPU-메인보드 소켓, 메모리 규격, 파워 용량, 케이스 폼팩터와 같은 규칙을 사용한다. 상세 기준은 기존 **admin/docs/compatibility-logic.md**와 서비스 구현을 함께 참고해야 한다.

주요 데이터

- **QuoteEntity**: 회원, 견적 유형, 이름, 용도, 설명, 이미지, 합계
- **QuoteItemEntity**: 견적, 상품 ID, 수량, 당시 단가
- **QuoteType**: 사용자 견적/프리셋/AI 견적 구분
- **AssemblyType, PurposeTag**: 조립 방식과 사용 목적

리뷰 메모

- 장점: 견적 항목에 가격을 스냅샷으로 보관해 이후 상품 가격 변경과 분리한다.
- 장점: 직접 견적, 프리셋, AI 견적이 공통 엔티티를 재사용한다.
- 위험: 호환성 규칙이 **QuoteSelectionValidator**, **QuoteService**, **CartService**에 분산되어 동일 조합이 화면별로 다르게 평가될 가능성이 있다. 단일 호환성 도메인 서비스가 필요하다.

2-3. 추천 프리셋 / AI 추천

추천 프리셋

/preset은 관리자 제작 **PRESET** 견적을 목적별로 보여주고 **/preset/{id}**에서 구성품, 가격, 설명을 표시한다. 로그인 사용자는 기존 프리셋을 자기 장바구니에 견적 묶음으로 추가할 수 있다. 프리셋 자체는 관리자 소유지만 장바구니는 회원별로 분리된다.

AI 추천

/ai-quotes에서 예산, 용도, 요구사항을 입력한다. **AiRecommendationController**가 요청을 받고 **GeminiRecommendationService**가 시스템 프롬프트와 상품 카탈로그를 조합하여 Gemini를 호출한다. **AiQuoteParser**가 모델 응답을 구조화된 추천 DTO로 변환하고 **AiQuoteService**가 실제 DB 상품과 매칭하여 AI 견적 및 추천 로그를 저장한다.

AI 결과에는 추천 부품, 총액, 요약, 장점, 키워드 등이 포함된다. **AiKeywordCleanupRunner**는 시작 시 상품명/브랜드/사양에 해당하는 금지 키워드를 AI 강조 키워드에서 제거한다.

실패 처리와 제약

- API 키가 없으면 AI 기능이 비활성 또는 오류 상태가 된다.

- 모델이 규격과 다른 텍스트를 반환하면 파서가 결과를 만들지 못할 수 있다.
- 모델 추천 상품은 최종적으로 DB 상품과 매칭되어야 건적으로 저장된다.
- AI 응답은 참고용이며 재고·가격·호환성은 서버의 현재 데이터로 재검증해야 한다.

리뷰 메모

- 장점: 프롬프트/상품 CSV를 리소스로 분리하고 AI 원문 응답을 저장해 추적성을 확보했다.
- 위험: 정적 CSV 상품 카탈로그와 DB 상품이 이중 관리된다. 가격·재고·판매 상태가 어긋날 수 있으므로 DB에서 카탈로그를 생성하는 방식이 안전하다.
- 위험: AI 로그에 사용자의 자유 입력 요구사항과 전체 응답을 저장한다. 개인정보/민감정보 필터링, 보존 기간, 관리자 열람 감사가 필요하다.
- 위험: 외부 AI 호출의 타임아웃, 재시도, 비용 제한, 사용자별 속도 제한 정책을 명시적으로 운영해야 한다.

2-4. 장바구니

기능 설명

장바구니는 개별 상품과 건적 묶음을 별도 엔티티로 관리하고 한 화면에서 합산한다.

- 개별 상품: 같은 상품을 다시 추가하면 수량을 합치고 재고 범위로 제한
- 건적 묶음: 구성품과 건적 수량을 함께 표시
- 공통: 선택/해제, 수량 변경, 삭제, 선택 금액 합계
- 주문 이동: 선택된 개별 상품과 건적만 주문서로 전달

CartService는 항상 로그인 아이디로 회원 ID를 찾고, 변경 대상 항목이 그 회원 장바구니에 속하는지 확인한다. 건적 장바구니를 표시할 때 구성품을 다시 읽어 호환성 문제 수를 계산한다.

주요 API

- GET /api/cart
- POST /api/cart/items
- PATCH /api/cart/items/{productId}/quantity
- PATCH /api/cart/items/{productId}/selection
- DELETE /api/cart/items/{productId}
- POST /api/cart/quotes, POST /api/cart/quotes/{quoteId}
- PATCH /api/cart/quotes/{quoteId}/selection
- DELETE /api/cart/quotes/{quoteId}

리뷰 메모

- 장점: 서비스 메시드가 소유권을 검증하여 ID 변조 공격을 방지한다.

- 위험: 동시에 같은 상품 수량을 변경할 때 마지막 쓰기 우선으로 덮어쓸 수 있다. 낙관적 잠금 버전 필드가 보이지 않는다.
- 위험: 장바구니 가격은 표시 시점과 결제 시점에 달라질 수 있다. 주문 생성 화면에서 가격 변경 안내 정책이 필요하다.

2-5. 주문·결제(연결 기능)

사용자가 `/orders/new`에서 배송지와 선택 상품을 확인하고 **POST** `/api/orders`로 대기 주문을 만든다. 주문은 개별 상품과 견적 묶음을 주문 그룹/항목 스냅샷으로 변환한다. `/payments/success`에서 Toss의 **paymentKey**, **orderId**, **amount**를 검증하고 서버가 Toss 승인 API를 호출한다.

승인 성공 후 결제 레코드 저장, 주문 결제 완료 전환, 재고 차감, 주문된 장바구니 항목 정리를 수행한다. 사용자는 주문 내역/상세를 보고 가능한 상태에서 취소나 반품을 요청한다. 취소는 전액 취소, 반품은 현재 구현상 절반 환불 정책이 포함돼 있다.

리뷰 메모

- 장점: 클라이언트 성공 화면만 신뢰하지 않고 서버가 주문 소유권과 결제 금액을 다시 검증한다.
- P0 후보: 결제 승인 같은 외부 API와 로컬 DB 트랜잭션은 원자적이지 않다. 외부 승인 후 DB 실패 시 결제만 완료될 수 있어 먹등키, 상태 머신, 보상 취소, 재처리 작업이 필요하다.
- P1: 주문번호가 당일 주문 건수 **count + 1**로 생성되어 동시 주문 시 충돌할 수 있다. DB 시퀀스/UUID/유니크 충돌 재시도를 사용해야 한다.
- P1: 재고 차감은 결제 승인 뒤 엔티티 값을 감소시키는 방식이므로 동시 결제에서 초과 판매 가능성을 검증해야 한다. 조건부 UPDATE 또는 비관/낙관 잠금이 필요하다.
- P1: “반품 요청 즉시 50% 환불”은 일반적인 주문 상태 모델과 다르며 금액 산정 근거가 없다. 관리자 승인, 반품 수거, 환불 금액 계산 단계를 분리해야 한다.

3. 관리자 페이지

3.1 접근 제어와 공통 구조

관리자 URL은 `/admin/**`이며 Spring Security의 `ROLE_ADMIN`만 접근할 수 있다. `AdminController` 하나가 모든 관리자 기능을 담당하고 Thymeleaf `templates/admin` 화면을 반환한다. 공통 메뉴는 `admin/fragments.html`, 스타일은 `admin.css`, `admin-form.css`, `admin-responsive.css`에 있다.

관리자 변경 요청은 POST와 CSRF 보호를 사용한다. 목록 페이지는 한 페이지 10개이며 컨트롤러가 전체 목록을 메모리에서 필터링한 뒤 잘라낸다.

3.2 대시보드

GET /admin

- 오늘 매출: 오늘 주문 중 결제 대기/취소 제외 합계
- 오늘 주문 수
- 오늘 신규 회원 수
- 재고 5개 미만 상품 수와 상위 5개
- 반품 요청 수와 상위 5개
- 최근 주문 5개

리뷰: 지표 정의가 코드에 직접 박혀 있다. 결제 취소/환불 금액, 타임존, 주문일과 결제일 차이를 반영하지 않아 재무 지표로 사용하기에는 부족하다. 전체 주문/상품을 읽어 애플리케이션에서 계산하므로 데이터 증가 시 집계 쿼리로 바뀌어야 한다.

3.3 회원 관리

목록/검색	GET /admin/members	키워드, 역할, 상태 필터 및 페이지 표시
상세	GET /admin/members/{id}	회원, 배송지, 주문 조회
수정	POST /admin/members/{id}	역할과 상태 변경

`AdminPrivacyMasking`은 이메일·전화·주소를 마스킹하는 보조 기능을 제공한다. 다만 모든 템플릿 출력에 일관되게 적용되는지 확인이 필요하다.

리뷰: 관리자가 자기 자신의 관리자 역할을 제거하거나 마지막 관리자를 비활성화할 수 있는 업무 규칙이 없다. 역할/상태 변경 이력과 수행 관리자 감사 로그도 없다. 목록은 `findAll()` 후 메모리 필터링하므로 DB 페이지 검색으로 전환해야 한다.

3.4 상품 관리

GET /admin/products는 등록/수정 품과 상품 목록을 함께 보여준다. **POST /admin/products**는 카테고리, 이름, 브랜드, 모델, 가격, 재고, 설명, 이미지, 판매 상태, 사양 키/값을 저장한다.

- 가격: 1,000원 이상, 100원 단위
- 재고: 음수 입력은 0으로 보정
- 사양: 키/값 목록을 JSON으로 변환
- 이미지: jpg/jpeg/png/webp/gif 확장자, UUID 파일명, **uploads/products** 저장

리뷰: 파일 확장자만 검사하고 실제 MIME/이미지 내용을 검사하지 않는다. 파일 크기 제한, 디코딩 검사, 저장소 격리, 악성 이미지 처리와 이전 이미지 삭제 정책이 필요하다. **salesStatus**가 문자열이어서 오타/미지원 상태가 저장될 가능성이 있다. enum과 Bean Validation을 권장한다.

3.5 추천 프리셋 관리

목록/목적 필터	GET /admin/presets
신규/수정 품	GET /admin/presets/new, GET /admin/presets/{id}/edit
저장	POST /admin/presets/save
삭제	POST /admin/presets/{id}/delete

관리자는 이름, 용도, 설명, 이미지와 카테고리별 상품/수량을 선택한다. **PresetAdminService**가 상품 존재 여부, 수량, 프리셋 유형을 확인하고 건적/항목을 저장한다.

리뷰: 수정 화면에서 같은 카테고리 항목이 복수일 때 **Collectors.toMap** 충돌 가능성이 있다. 입력 규칙으로 카테고리당 하나만 허용할지, 저장/조회 양쪽에서 명확히 검증해야 한다. 프리셋 삭제가 기존 회원 장바구니/주문과 어떤 관계인지 데이터베이스 FK 정책까지 문서화해야 한다.

3.6 주문 관리

목록	GET /admin/orders	주문번호/회원명, 상태, 반품 요청 필터
상세	GET /admin/orders/{orderNumber}	주문 그룹/품목, 회원, 결제 조회
상태 변경	POST /admin/orders/{orderNumber}/status	요청한 enum 상태로 즉시 변경

리뷰: 관리자 상태 변경이 허용 전이 규칙을 거치지 않고 **changeStatus**를 직접 호출한다. 배송 중→결제 대기 같은 역전, 취소 주문→배송 완료 같은 잘못된 상태가 가능할 수 있다. 결제 취소/환불, 재고 복원과도 연동되지 않으므로 전용 **AdminOrderService**에서 상태 전이와 부수 효과를 묶어야 한다.

3.7 AI 추천 로그

GET /admin/ai-logs가 추천 기록을 최신순으로 읽고 회원과 연결하며 응답 JSON을 보기 좋게 포맷한다. 모델 요청과 결과 분석, 장애 조사에 유용하다.

리뷰: 전체 로그와 전체 관련 회원을 한 번에 읽어 페이지가 커질수록 느려진다. 페이지네이션, 기간/회원/성공 여부 필터, 응답 크기 제한이 필요하다. 사용자 요구사항에 개인정보가 포함될 수 있으므로 마스킹·열람 권한·보존 및 삭제 정책이 필요하다.

3.8 관리자 코드 구조 개선안

현재 376줄 **AdminController**에 조회, 업무 규칙, 파일 저장, JSON 변환, 페이지징이 모여 있다. 다음과 같이 분리하는 것이 좋다.

- **AdminDashboardService**: 집계 쿼리와 지표 정의
- **AdminMemberService**: 역할/상태 전이, 마지막 관리자 보호, 감사 로그
- **AdminProductService** + **ImageStorageService**: 상품 검증과 안전한 파일 저장
- **AdminOrderService**: 상태 머신, 결제/재고 연동
- **AdminAiLogService**: 검색, 마스킹, 페이지 조회
- Spring Data **Pageable**: DB 수준 필터·정렬·페이징

4. 종합 코드 리뷰

4.1 잘된 점

- 도메인별 패키지와 Controller/Service/Repository 계층이 비교적 명확하다.
- Spring Security로 공개/회원/관리자 권한 경계를 중앙 관리한다.
- 회원 비밀번호와 이메일 인증번호를 BCrypt 해시로 저장한다.
- 장바구니·배송지·주문 조회에서 로그인 회원 소유권을 서비스 레벨에서 확인한다.
- 주문과 견적 품목에 가격/상품 정보를 스냅샷으로 남기는 방향이 적절하다.
- AI 요청/응답 기록과 프롬프트 리소스를 분리하여 추적 가능성을 확보했다.
- 관리자, 회원, OAuth 등 통합 테스트가 존재하고 기존 성공 결과도 다수 있다.

4.2 우선순위별 발견 사항

P0 — 즉시 확인

1. **한글 소스/화면 문자열 손상**: 다수 Java-HTML 문자열이 ?똥똥, 똥똥?? 형태로 깨져 있다. 기존 테스트도 기대 문자열과 렌더링 문자열 불일치로 실패한다. 파일 인코딩, Git 변환, DB/IDE/빌드 인코딩을 UTF-8로 통일하고 원본 한글을 복구해야 한다.
2. **결제 승인과 DB 반영 불일치 가능성**: Toss 승인 성공 후 로컬 트랜잭션이 실패하면 고객 결제만 승인될 수 있다. 결제 요청/승인 상태 저장, 멍등성, 보상 취소, 미완료 재처리 작업과 운영 알림이 필요하다.

P1 — 출시 전 보완

1. **동시 주문번호 충돌**: 당일 주문 수를 세어 +1하는 방식은 동시에 두 주문이 만들어질 때 같은 번호를 생성할 수 있다.
2. **재고 경합/초과 판매**: 재고 차감 시 동시성 제어가 명시적이지 않다. DB 조건부 차감 또는 잠금과 경합 테스트가 필요하다.
3. **관리자 주문 상태 무제한 변경**: 상태 전이 검증 없이 enum 값을 적용하며 결제/재고 부수 효과와 연결되지 않는다.
4. **업로드 검증 부족**: 파일 확장자만 허용 목록으로 검사한다. 크기, MIME, 실제 이미지 디코딩, 저장 위치/서빙 정책을 강화해야 한다.
5. **AI 개인정보/비용 통제**: 자유 입력과 응답 전문 저장에 대한 필터, 보존 기간, 속도/예산 제한, 감사 정책이 없다.
6. **설정 기본 비밀번호**: `spring.datasource.password=${DB_PASSWORD:MULTI}`는 운영에서 환경 변수 누락을 숨긴다. 기본값을 없애 시작 실패로 드러내야 한다.
7. **회원 이메일 변경 재인증**: 계정 복구에 쓰는 이메일 변경 시 새 주소 확인을 강제해야 한다.

P2 — 유지보수와 성능

1. 관리자 목록/대시보드/AI 로그가 **findAll()** 후 메모리 필터·집계를 수행한다. Repository 조건 검색, 집계 쿼리, **Pageable**로 전환한다.
2. 호환성 규칙이 견적과 장바구니에 흩어져 있다. 단일 서비스와 표 기반 테스트로 통합한다.
3. **specJson** 파싱 예외를 무시한다. 상품 ID와 원인을 구조화 로그로 남기고 관리자 저장 시 차단한다.
4. 정적 AI 상품 CSV와 DB 상품을 이중 관리한다. 요청 시 DB 판매 가능 상품으로 카탈로그를 생성하거나 동기화 버전을 검증한다.
5. **AdminController**, **CartService**, **QuoteService**, **OrderService**가 크고 책임이 많다. 조회 조립, 업무 규칙, 외부 연동을 분리한다.
6. 탈퇴·AI 로그·주문 개인정보의 보존/파기 정책과 코드가 연결돼 있지 않다.

P3 — 코드 품질

1. 일부 Java 파일 전체가 한 줄로 되어 있다. 표준 포매터를 적용한다.
2. 사용자 메시지가 컨트롤러에 흩어져 있고 현재 문자 손상까지 있다. 메시지 번들과 UTF-8 리소스로 분리한다.
3. 문자열 상태(**salesStatus**)를 enum/값 객체로 바꾼다.
4. 무시된 예외에 최소한의 진단 로그를 추가한다.

4.3 테스트 상태

기존 **target/surefire-reports** 기준:

AdminPageIntegrationTests	10 성공
CompickApplicationTests	9 성공
GoogleOAuthIntegrationTests	3 성공
MemberServiceIntegrationTests	8 성공
MemberIntegrationTests	8개 중 4개 실패

실패 4건은 로그인 페이지 문자열, 가입 리다이렉트, 홈 알림 문자열, 중복 확인 JSON 응답 계약과 관련된다. 단순히 테스트가 오래된 것인지 구현 회귀인지 요구사항을 기준으로 판단해야 하지만, 현재 상태에서 전체 테스트 통과라고 볼 수 없다.

4.4 권장 추가 테스트

- 회원: 이메일 인증 만료/재사용/5회 실패/동시 가입, 이메일 변경 인증, 정지 회원 로그인
- 권한: 다른 회원 배송지·장바구니·주문 ID 접근, 일반 사용자의 관리자 URL 접근
- 상품: 잘못된 사양 JSON, 품질/판매중지 상품, 악성·대용량 업로드
- 견적: 카테고리 중복/필수 부품 누락, 소켓·메모리·파워·폼팩터 조합 표 테스트
- 장바구니: 동시에 수량 변경, 가격/재고 변경 후 주문 이동
- 결제: 중복 콜백, 금액 위조, Toss 성공 후 DB 실패, 승인 타임아웃 후 재조회, 취소 실패 보상

- 주문: 동시에 주문번호 생성, 상태 전이 표, 반품/환불 금액과 재고 복원
- 관리자: 마지막 관리자 보호, 잘못된 주문 상태 전이, 대량 데이터 페이지 성능

4.5 권장 개선 순서

1. UTF-8 한글 복구 후 전체 테스트의 기대 계약을 확정한다.
2. 결제 멍등성, 주문번호, 재고 동시성을 먼저 보강한다.
3. 관리자 주문 상태 머신과 결제/재고 연동을 서비스로 이동한다.
4. 파일 업로드와 AI 로그 개인정보 정책을 강화한다.
5. 관리자 DB 페이징/집계와 대형 서비스 분리를 진행한다.
6. 호환성 규칙을 단일화하고 회귀 테스트를 확대한다.

5. API·화면·파일 맵

회원

	URL	Controller/Service/View
로그인	GET/POST /login	MemberController, SecurityConfig, member/login.html
가입	GET/POST /members/signup	MemberController, MemberService, member/signup.html
중복 확인	GET /api/members/check-login-id, /check-email	MemberApiController
이메일 인증	POST /api/email-verifications/send, /confirm	EmailVerificationApiController/Service
Google 로그인	/oauth2/authorization/google	CompickOidcUserService, SocialAccountService
계정 복구	/members/find-id/, /members/password-reset/	RecoveryController, 복구 템플릿
마이페이지	GET /mypage	MemberController, mypage/index.html
프로필/비밀번호/탈퇴	/mypage/profile, /mypage/password, /mypage/withdraw	MemberController, MemberService
배송지	/mypage/addresses/, /api/addresses/	AddressController, AddressApiController, AddressService

구매

	URL	Controller/Service/View
상품 목록	GET /products	ShoppingPageController, shopping/products.html
상품 API	GET /api/products/**	ProductApiController, ProductService
직접 견적	GET /quotes/new	QuotePageController, quote-new.html, quote-builder.js
프리셋	GET /preset, /preset/{id}	QuotePageController, QuoteService, preset 템플릿
AI 견적	GET/POST /ai-quotes	AiRecommendationController, Gemini/Parser/AiQuote 서비스
장바구니	GET /cart, /api/cart/**	Cart 컨트롤러/서비스, cart/index.html, cart.js

기능	URL	담당 기능/파일
주문서/목록/상세	GET /orders/new, /orders, /orders/{no}	Order 페이지 컨트롤러/서비스, order 템플릿
주문 API	POST /api/orders, 취소/반품 URL	OrderApiController, OrderService
결제 결과	GET /payments/success, /fail	PaymentController, PaymentService, TossPaymentService

관리자

기능	URL	담당 파일
대시보드	GET /admin	admin/dashboard.html
회원	GET /admin/members, 상세/수정 URL	member-list.html, member-detail.html
상품	GET/POST /admin/products	product-form.html
프리셋	/admin/presets/**	preset-list.html, preset-form.html
주문	/admin/orders/**	order-list.html, order-detail.html
AI 로그	GET /admin/ai-logs	ai-log-list.html

주요 정적 파일

파일명	기능
static/js/products.js	상품 검색, 필터, 목록 갱신
static/js/product-modal.js	상품 상세 모달
static/js/quote-builder.js	직접 견적 구성
static/shopping/js/ai-recommendation.js	AI 추천 요청/표시
static/js/cart.js	장바구니 수량·선택·삭제
static/js/order.js, order-detail.js	주문 생성, 취소/반품
static/js/member.js	가입/회원 폼 상호작용
static/js/address*.js	배송지 목록/폼 처리